

CSU33032 Project 1: Forward Web Proxy

Author: Joel Mathew Jojan (23376190)

Submission: Protocol design, implementation description, and annotated code listing (single PDF).

1. High-Level Protocol Design and Implementation

1.1 Role of the Proxy

The forward web proxy sits between clients (e.g. browsers) and origin servers. The client configures its HTTP/HTTPS proxy to point to the proxy's host and port (e.g. `localhost:8080`). All HTTP and HTTPS requests from the client are then sent to the proxy instead of directly to the target server. The proxy parses each request, decides whether to block it, serve it from cache, or forward it (or tunnel it for HTTPS), and returns a response to the client.

1.2 Protocol Behaviour

HTTP (non-CONNECT) requests

- The client sends a request in absolute form, e.g. `GET http://example.com/path HTTP/1.1`, or in origin form with a `Host` header. The proxy parses the request line and headers to obtain method, target host, port, and path.
- The proxy checks a **block list** (by host and optional path prefix). If the request is blocked, it replies with `403 Forbidden` and does not contact the origin.
- For **GET** requests only, the proxy looks up an in-memory **response cache** using a key derived from host, port, and path. On a cache hit (and non-expired entry), it sends the cached response back to the client and does not contact the origin.
- Otherwise the proxy opens a TCP connection to the origin server, sends an HTTP request in origin form (method, path, version, and forwarded headers, with `Connection: close`), reads the full response (status line, headers, body), and sends that response back to the client. For GET requests with a cacheable 200 response (no `no-store/private`), the proxy stores the response in the cache with a TTL derived from `Cache-Control` or `Expires`, or a default.

HTTPS (CONNECT) requests

- The client sends `CONNECT host:port HTTP/1.1`. The proxy again checks the block list; if blocked, it responds with `403 Forbidden`.
- If not blocked, the proxy opens a TCP connection to the given host and port, sends `HTTP/1.1 200 Connection Established` plus a blank line to the client, then **tunnels** raw bytes bidirectionally between the client and the origin. The proxy does not interpret the tunnelled data (TLS is end-to-end between client and origin). No caching is applied to CONNECT.

Block list

- Rules are either a host (e.g. `example.com`) or host plus path prefix (e.g. `example.com/ads`). Input is normalised (e.g. strip `http://https://` and trailing slashes). A request is blocked if its host matches a host-only rule (including subdomains, e.g. `www.example.com` for rule `example.com`) or if host and path match a host+path rule.

Caching

- Only HTTP GET requests are cached; CONNECT (HTTPS) is never cached. Only 200 responses are stored. Responses with **Cache-Control: no-store** or **private** are not cached. TTL is taken from **max-age**, **Expires**, or a default (e.g. 5 minutes). The cache is bounded (e.g. 500 entries) with LRU eviction. Cache key is **host|port|path**.

1.3 Implementation Overview

- **ProxyServer:** Listens on a configurable port, accepts TCP connections, and for each connection starts a new thread running a **ClientHandler**.
- **ClientHandler:** For each client socket, reads the request line and headers, derives host/port/path, checks the block list, then either responds 403, serves from cache (GET only), runs the CONNECT tunnel, or forwards the request to the origin and parses the response. It records timing and source (Cache / Origin / Blocked / Tunnel) and notifies a **RequestListener**.
- **BlockList:** Thread-safe set of normalised rules; supports host and host+path matching with subdomain support.
- **ResponseCache:** Thread-safe map of cache key to **CacheEntry** (status line, headers, body, expiry). GET responses are stored after forwarding; TTL and eviction (expired first, then LRU) are applied.
- **HttpResponseReader:** Reads status line and headers line-by-line from the origin stream, then body via **Content-Length** or **Transfer-Encoding: chunked**, and returns a single **Response** object.
- **ManagementConsole:** Swing UI implementing **RequestListener**; shows a request log (with time, method, host, path, client, status, time in ms, source), a block list (add/remove rules), and a cache view (refresh on a timer).

Design choices: one thread per client for simplicity; shared block list and cache for all handlers (thread-safe); CONNECT implemented as a blind byte tunnel; cache only for HTTP GET to keep semantics clear and avoid caching encrypted HTTPS content.

2. Code Listing with Meaningful Comments

The following sections contain the full source code of the project, with comment blocks inserted to explain the role of each class and the main steps inside the code.

2.1 ProxyServer.java

Entry point: creates the block list, response cache, and management console; starts the proxy on a given port and runs the accept loop. Each accepted connection is handled in a new thread.

```
// Main entry point: listens for TCP connections and hands each client to a ClientHandler thread  
// Optionally uses a management console, block list, and response cache.
```

```
import javax.swing.*;  
import java.io.IOException;
```

```

import java.net.ServerSocket;
import java.net.Socket;

public class ProxyServer {

    private static final int DEFAULT_PORT = 8080;

    private final int port;
    private final RequestListener requestListener;
    private final BlockList blockList;
    private final ResponseCache responseCache;

    // Constructor chain: allows optional listener, block list, and cache (null = use defaults
    public ProxyServer(int port) {
        this(port, null, null, null);
    }

    public ProxyServer(int port, RequestListener requestListener) {
        this(port, requestListener, null, null);
    }

    public ProxyServer(int port, RequestListener requestListener, BlockList blockList) {
        this(port, requestListener, blockList, null);
    }

    public ProxyServer(int port, RequestListener requestListener, BlockList blockList, ResponseCache responseCache) {
        this.port = port;
        this.requestListener = requestListener;
        this.blockList = blockList;
        this.responseCache = responseCache != null ? responseCache : new ResponseCache();
    }

    // Accept loop: one thread per client; handler runs until the request/response (or tunnel)
    public void start() throws IOException {
        try (ServerSocket serverSocket = new ServerSocket(port)) {
            System.out.println("ProxyServer listening on Port: " + port + "\n");
            while (true) {
                Socket clientSocket = serverSocket.accept();
                System.out.println("Accepted connection from: " + clientSocket.getRemoteSocketAddress());
                ClientHandler handler = new ClientHandler(clientSocket, requestListener, blockList, responseCache);
                new Thread(handler).start();
            }
        }
    }

    // main: parse port from args, create shared block list and cache, show console on EDT, then start
    public static void main(String[] args) {

```

```

int port = args.length == 1 ? Integer.parseInt(args[0]) : DEFAULT_PORT;
BlockList blockList = new BlockList();
ResponseCache responseCache = new ResponseCache();
ManagementConsole console = new ManagementConsole(blockList, responseCache);
SwingUtilities.invokeLater(() -> console.setVisible(true));
try {
    new ProxyServer(port, console, blockList, responseCache).start();
} catch (IOException e) {
    System.err.println("Failed to start Proxy Server: " + e.getMessage());
    e.printStackTrace();
}
}
}

```

2.2 RequestListener.java

Callback interface used by the proxy to report each completed request (including timing and source) to the UI.

```

// Callback interface for the proxy to report each completed request (with timing and source).

public interface RequestListener {
    void onRequest(RequestRecord record);
}

```

2.3 RequestRecord.java

Immutable value object holding one proxied request: timestamp, method, host, port, path, client address, blocked flag, duration in ms, and source (Cache / Origin / Blocked / Tunnel). Used for logging and UI display.

```

// Immutable value object for one proxied request: method, host, path, client, status, duration

import java.time.Instant;
import java.time.ZoneId;
import java.time.format.DateTimeFormatter;

public class RequestRecord {

    private static final DateTimeFormatter TIME_FMT =
        DateTimeFormatter.ofPattern("HH:mm:ss.SSS").withZone(ZoneId.systemDefault());

    private static final int PORT_HTTP = 80;
    private static final int PORT_HTTPS = 443;

    public static final String SOURCE_CACHE = "Cache";

```

```

public static final String SOURCE_ORIGIN = "Origin";
public static final String SOURCE_BLOCKED = "Blocked";
public static final String SOURCE_TUNNEL = "Tunnel";

private final Instant timestamp;
private final String method;
private final String host;
private final int port;
private final String path;
private final String clientAddress;
private final boolean blocked;
private final Long durationMs;
private final String source;

public RequestRecord(Instant timestamp, String method, String host, int port, String path,
    this(timestamp, method, host, port, path, clientAddress, false, null, null);
}

public RequestRecord(Instant timestamp, String method, String host, int port, String path,
    this(timestamp, method, host, port, path, clientAddress, blocked, null, null);
}

public RequestRecord(Instant timestamp, String method, String host, int port, String path,
    boolean blocked, Long durationMs, String source) {
    this.timestamp = timestamp;
    this.method = method;
    this.host = host;
    this.port = port;
    this.path = path;
    this.clientAddress = clientAddress;
    this.blocked = blocked;
    this.durationMs = durationMs;
    this.source = source;
}

public Instant getTimestamp() { return timestamp; }
public String getMethod() { return method; }
public String getHost() { return host; }
public int getPort() { return port; }
public String getPath() { return path; }
public String getClientAddress() { return clientAddress; }
public boolean isBlocked() { return blocked; }
public Long getDurationMs() { return durationMs; }
public String getSource() { return source; }

public String getTimeString() {
    return TIME_FMT.format(timestamp);
}

```

```

    }

    // Display host:port, omitting :80 and :443 when standard.
    public String getHostPort() {
        return (port == PORT_HTTP || port == PORT_HTTPS) ? host : host + ":" + port;
    }
}

```

2.4 BlockList.java

Thread-safe block list. Rules are stored in normalised form (host or host/path). Full URLs are normalised by stripping scheme and trailing slashes. Matching supports subdomains (e.g. www.example.com matches rule example.com).

// Thread-safe block list for URL/host rules. Supports full URLs (normalised to host or host/path) and subdomain matching. Used by the proxy to deny requests before forwarding.

```

import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;

public class BlockList {

    private static final String PREFIX_HTTPS = "https://";
    private static final String PREFIX_HTTP = "http://";

    private final Set<String> rules = ConcurrentHashMap.newKeySet();

    // Strip scheme and trailing slashes; return host or host/path for storage.
    public static String normalizeRule(String input) {
        if (input == null) return null;
        String s = input.trim().toLowerCase();
        if (s.isEmpty()) return null;
        if (s.startsWith(PREFIX_HTTPS)) s = s.substring(PREFIX_HTTPS.length());
        else if (s.startsWith(PREFIX_HTTP)) s = s.substring(PREFIX_HTTP.length());
        if (s.isEmpty()) return null;
        while (s.endsWith("/") && s.length() > 1) s = s.substring(0, s.length() - 1);
        return s;
    }

    public String add(String rule) {
        String normalized = normalizeRule(rule);
        if (normalized == null) return null;
        rules.add(normalized);
        return normalized;
    }
}

```

```

public void remove(String rule) {
    if (rule == null) return;
    String normalized = normalizeRule(rule);
    if (normalized != null) rules.remove(normalized);
    rules.remove(rule.trim().toLowerCase());
}

public boolean contains(String rule) {
    if (rule == null) return false;
    String normalized = normalizeRule(rule);
    return normalized != null && rules.contains(normalized);
}

// Exact match or subdomain: requestHost equals ruleHost or ends with .ruleHost.
private static boolean hostMatches(String requestHost, String ruleHost) {
    if (requestHost == null || ruleHost == null) return false;
    if (requestHost.equals(ruleHost)) return true;
    return requestHost.endsWith(".") + ruleHost);
}

// Check all rules: host-only rules match by host (with subdomain); host/path rules match p
public boolean isBlocked(String host, String path) {
    if (host == null) return false;
    String h = host.toLowerCase();
    String p = (path == null || path.isEmpty()) ? "/" : path;
    if (!p.startsWith("/")) p = "/" + p;
    for (String rule : rules) {
        int slash = rule.indexOf('/');
        if (slash < 0) {
            if (hostMatches(h, rule)) return true;
        } else {
            String ruleHost = rule.substring(0, slash);
            String rulePath = "/" + rule.substring(slash + 1);
            if (hostMatches(h, ruleHost) && (rulePath.equals("/") || p.startsWith(rulePath)))
                return true;
        }
    }
    return false;
}

public String[] getAll() {
    return rules.toArray(new String[0]);
}
}

```

2.5 CacheEntry.java

Single cached HTTP response: status line, list of header lines, body bytes, expiry time, and last-access time for LRU. Body is stored after decoding chunked encoding so it can be re-sent with Content-Length.

```
// Single cached HTTP response: status line, headers, body, expiry and last-access for LRU.

import java.util.List;

public class CacheEntry {

    private final String statusLine;
    private final List<String> headers;
    private final byte[] body;
    private final long expiryMillis;
    private volatile long lastAccessMillis;

    public CacheEntry(String statusLine, List<String> headers, byte[] body, long expiryMillis) {
        this.statusLine = statusLine;
        this.headers = headers;
        this.body = body;
        this.expiryMillis = expiryMillis;
        this.lastAccessMillis = System.currentTimeMillis();
    }

    public String getStatusLine() { return statusLine; }
    public List<String> getHeaders() { return headers; }
    public byte[] getBody() { return body; }
    public long getExpiryMillis() { return expiryMillis; }
    public long getLastAccessMillis() { return lastAccessMillis; }

    public void touch() {
        this.lastAccessMillis = System.currentTimeMillis();
    }

    public boolean isExpired() {
        return System.currentTimeMillis() > expiryMillis;
    }
}
```

2.6 ResponseCache.java

In-memory cache keyed by host|port|path. GET responses (200, cacheable) are stored with TTL from Cache-Control/Expires or default. no-store/private responses are not stored. Eviction: when over capacity, remove expired entries first, then LRU. CacheEntryInfo is used by the UI to show URL, size, expires, last access.

```

// In-memory HTTP response cache for GET requests. Bounded, thread-safe, LRU eviction.
// TTL from Cache-Control/Expires or default; no-store/private responses are not stored.

import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

public class ResponseCache {

    private static final int DEFAULT_MAX_ENTRIES = 500;
    private static final long DEFAULT_TTL_SECONDS = 300;
    private static final long NO_CACHE_TTL_SECONDS = 60;
    private static final String CACHE_CONTROL = "cache-control";
    private static final String EXPIRES = "expires";
    private static final String MAX_AGE_PREFIX = "max-age=";

    private final ConcurrentHashMap<String, CacheEntry> cache = new ConcurrentHashMap<>();
    private final int maxEntries;
    private final long defaultTtlMillis;

    public ResponseCache() {
        this(DEFAULT_MAX_ENTRIES, DEFAULT_TTL_SECONDS);
    }

    public ResponseCache(int maxEntries, long defaultTtlSeconds) {
        this.maxEntries = maxEntries;
        this.defaultTtlMillis = defaultTtlSeconds * 1000;
    }

    public static String cacheKey(String host, int port, String path) {
        return host + "|" + port + "|" + (path != null ? path : "/");
    }

    // Return entry only if present and not expired; update last-access on hit.
    public CacheEntry get(String key) {
        CacheEntry entry = cache.get(key);
        if (entry == null || entry.isExpired()) return null;
        entry.touch();
        return entry;
    }

    public void put(String key, CacheEntry entry) {
        cache.put(key, entry);
        evictIfNeeded();
    }
}

```

```

// Sort by expired first, then by oldest last-access; remove until size <= maxEntries.
private void evictIfNeeded() {
    if (cache.size() <= maxEntries) return;
    List<Map.Entry<String, CacheEntry>> entries = new ArrayList<>(cache.entrySet());
    entries.sort((a, b) -> {
        boolean aExp = a.getValue().isExpired();
        boolean bExp = b.getValue().isExpired();
        if (aExp != bExp) return aExp ? -1 : 1;
        return Long.compare(a.getValue().getLastAccessMillis(), b.getValue().getLastAccessM
    });
    int toRemove = cache.size() - maxEntries;
    for (int i = 0; i < toRemove && i < entries.size(); i++) {
        cache.remove(entries.get(i).getKey());
    }
}

// Return expiry time in ms; 0 means do not store. Uses max-age, Expires, or default TTL.
public long computeExpiry(Map<String, String> responseHeaders) {
    String cacheControl = responseHeaders.get(CACHE_CONTROL);
    if (cacheControl != null) {
        String lower = cacheControl.toLowerCase();
        if (lower.contains("no-store") || lower.contains("private")) return 0;
        if (lower.contains("no-cache")) return System.currentTimeMillis() + NO_CACHE_TTL_SECONDS;
        int maxAge = parseMaxAge(cacheControl);
        if (maxAge >= 0) return System.currentTimeMillis() + maxAge * 1000L;
    }
    String expires = responseHeaders.get(EXPIRES);
    if (expires != null) {
        try {
            long exp = java.time.ZonedDateTime.parse(expires,
                java.time.format.DateTimeFormatter.RFC_1123_DATE_TIME).toInstant().toEpochSecond() * 1000;
            if (exp > System.currentTimeMillis()) return exp;
        } catch (Exception ignored) {}
    }
    return System.currentTimeMillis() + defaultTtlMillis;
}

private static int parseMaxAge(String cacheControl) {
    String lower = cacheControl.toLowerCase();
    int idx = lower.indexOf(MAX_AGE_PREFIX);
    if (idx < 0) return -1;
    int start = idx + MAX_AGE_PREFIX.length();
    int end = start;
    while (end < cacheControl.length()
        && (Character.isDigit(cacheControl.charAt(end)) || cacheControl.charAt(end) == '-'))
        end++;
    try {
        return Integer.parseInt(cacheControl.substring(start, end).trim());
    } catch (NumberFormatException ignored) {}
}

```

```

    } catch (NumberFormatException e) {
        return -1;
    }
}

public int size() { return cache.size(); }

// Snapshot for UI: key, body size, expiry, last access; getDisplayUrl formats key as host.
public static class CacheEntryInfo {
    private final String key;
    private final int bodySize;
    private final long expiryMillis;
    private final long lastAccessMillis;

    public CacheEntryInfo(String key, int bodySize, long expiryMillis, long lastAccessMillis) {
        this.key = key;
        this.bodySize = bodySize;
        this.expiryMillis = expiryMillis;
        this.lastAccessMillis = lastAccessMillis;
    }

    public String getKey() { return key; }
    public int getBodySize() { return bodySize; }
    public long getExpiryMillis() { return expiryMillis; }
    public long getLastAccessMillis() { return lastAccessMillis; }

    public String getDisplayUrl() {
        if (key == null) return "";
        int first = key.indexOf('|');
        int second = key.indexOf('|', first + 1);
        if (first < 0 || second < 0) return key;
        String hostPart = key.substring(0, first);
        String portPart = key.substring(first + 1, second);
        String pathPart = key.substring(second + 1);
        return hostPart + ":" + portPart + " " + pathPart;
    }
}

public List<CacheEntryInfo> getSnapshot() {
    List<CacheEntryInfo> list = new ArrayList<>();
    for (Map.Entry<String, CacheEntry> e : cache.entrySet()) {
        CacheEntry ent = e.getValue();
        list.add(new CacheEntryInfo(e.getKey(), ent.getBody().length, ent.getExpiryMillis(), ent.getLastAccessMillis()));
    }
    list.sort((a, b) -> Long.compare(b.getLastAccessMillis(), a.getLastAccessMillis()));
    return list;
}

```

```
}
```

2.7 `HttpServletResponseReader.java`

Parses an HTTP response from an `InputStream`: reads status line and header lines until a blank line, then body. Body is read using `Content-Length`, or `Transfer-Encoding: chunked` (decode to a single byte array), or read until connection close. Used when forwarding HTTP requests to the origin.

// Parses an HTTP response from a stream: status line, headers, body (Content-Length or chunked)

```
import java.io.ByteArrayOutputStream;
import java.io.IOException;
import java.io.InputStream;
import java.nio.charset.StandardCharsets;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class HttpServletResponseReader {

    private static final int CHUNKED_MARKER = -2;
    private static final int BUFFER_SIZE = 8192;
    private static final String CHARSET = StandardCharsets.ISO_8859_1.name();
    private static final String TRANSFER_ENCODING = "transfer-encoding";
    private static final String CONTENT_LENGTH = "content-length";

    public static class Response {
        public final String statusLine;
        public final List<String> headerLines;
        public final Map<String, String> headers;
        public final byte[] body;

        Response(String statusLine, List<String> headerLines, Map<String, String> headers, byte[] body) {
            this.statusLine = statusLine;
            this.headerLines = headerLines;
            this.headers = headers;
            this.body = body;
        }
    }

    // Read until \n; strip \r. Used for status and header lines.
    public static String readLine(InputStream in) throws IOException {
        ByteArrayOutputStream buf = new ByteArrayOutputStream();
        int b;
```

```

        while ((b = in.read()) != -1) {
            if (b == '\n') break;
            if (b != '\r') buf.write(b);
        }
        return buf.toString(CHARSET);
    }

    // Read status, then headers until blank line, then body (length from Content-Length or chunked transfer encoding)
    public static Response readResponse(InputStream in) throws IOException {
        String statusLine = readLine(in);
        if (statusLine == null || statusLine.isEmpty()) return null;
        List<String> headerLines = new ArrayList<>();
        Map<String, String> headers = new HashMap<>();
        String line;
        while ((line = readLine(in)) != null && !line.isEmpty()) {
            headerLines.add(line);
            int colon = line.indexOf(':');
            if (colon > 0) {
                String key = line.substring(0, colon).trim().toLowerCase();
                String value = line.substring(colon + 1).trim();
                headers.put(key, value);
            }
        }
        int contentLength = resolveBodyLength(headers);
        byte[] body = readBody(in, contentLength);
        return new Response(statusLine, headerLines, headers, body);
    }

    private static int resolveBodyLength(Map<String, String> headers) {
        String te = headers.get(TRANSFER_ENCODING);
        if (te != null && te.toLowerCase().contains("chunked")) return CHUNKED_MARKER;
        String cl = headers.get(CONTENT_LENGTH);
        if (cl != null) {
            try { return Integer.parseInt(cl.trim()); }
            catch (NumberFormatException e) { return -1; }
        }
        return -1;
    }

    private static byte[] readBody(InputStream in, int contentLength) throws IOException {
        if (contentLength == 0) return new byte[0];
        if (contentLength > 0) return readExactly(in, contentLength);
        if (contentLength == CHUNKED_MARKER) return readChunkedBody(in);
        return readUntilClose(in);
    }

    private static byte[] readExactly(InputStream in, int n) throws IOException {

```

```

byte[] buf = new byte[n];
int off = 0;
while (off < n) {
    int r = in.read(buf, off, n - off);
    if (r <= 0) break;
    off += r;
}
return buf;
}

// Chunked: each chunk is size (hex) then that many bytes; size 0 ends.
private static byte[] readChunkedBody(InputStream in) throws IOException {
    ByteArrayOutputStream out = new ByteArrayOutputStream();
    while (true) {
        String sizeLine = readLine(in);
        if (sizeLine == null) break;
        int semicolon = sizeLine.indexOf(';');
        String sizeStr = semicolon >= 0 ? sizeLine.substring(0, semicolon).trim() : sizeLine;
        int chunkSize;
        try { chunkSize = Integer.parseInt(sizeStr, 16); }
        catch (NumberFormatException e) { break; }
        if (chunkSize == 0) break;
        byte[] chunk = readExactly(in, chunkSize);
        out.write(chunk);
        readLine(in);
    }
    return out.toByteArray();
}

private static byte[] readUntilClose(InputStream in) throws IOException {
    ByteArrayOutputStream out = new ByteArrayOutputStream();
    byte[] buf = new byte[BUFFER_SIZE];
    int n;
    while ((n = in.read(buf)) != -1) out.write(buf, 0, n);
    return out.toByteArray();
}
}

```

2.8 ClientHandler.java

Handles one client connection. Flow: (1) Read request line and headers; (2) Parse target into host, port, path (CONNECT vs absolute URL vs origin-form); (3) If blocked → 403 and SOURCE_BLOCKED; (4) If GET and cache hit → send cached response, SOURCE_CACHE; (5) If CONNECT → tunnel bytes both ways, SOURCE_TUNNEL; (6) Else forward to origin, send response, SOURCE_ORIGIN, and optionally store GET 200 in cache. Helper methods: readHeaders, sendBlockedResponse, runConnectTunnel, copyStream, fetchFromOrigin, writeForwardRequest,

maybeStoreInCache, isSuccessStatus, normalizeHeadersForCache.

*// Handles one client connection: parses request, checks block list and cache, forwards or tunnels
// Reports each completed request (with duration and source) to the RequestListener.*

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.io.OutputStream;
import java.net.Socket;
import java.nio.charset.StandardCharsets;
import java.time.Instant;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.Iterator;
import java.util.List;
import java.util.Map;

public class ClientHandler implements Runnable {

    private static final int CONNECT_DEFAULT_PORT = 443;
    private static final int HTTP_DEFAULT_PORT = 80;
    private static final int ORIGIN_SO_TIMEOUT_MS = 15000;
    private static final int TUNNEL_BUFFER_SIZE = 8192;
    private static final String HTTP_PREFIX = "http://";
    private static final String HTTPS_PREFIX = "https://";
    private static final int HTTPS_PREFIX_LEN = 8;
    private static final int HTTP_PREFIX_LEN = 7;
    private static final String CONNECTION_ESTABLISHED = "HTTP/1.1 200 Connection Established\r\n";
    private static final String FORBIDDEN_BODY = "Blocked by proxy.";
    private static final String STATUS_CODE_200 = "200";

    private final Socket clientSocket;
    private final RequestListener requestListener;
    private final BlockList blockList;
    private final ResponseCache responseCache;

    public ClientHandler(Socket clientSocket) {
        this(clientSocket, null, null, null);
    }

    public ClientHandler(Socket clientSocket, RequestListener requestListener) {
        this(clientSocket, requestListener, null, null);
    }

    public ClientHandler(Socket clientSocket, RequestListener requestListener, BlockList blockList) {
        this(clientSocket, requestListener, blockList, null);
    }
}
```

```

    }

    public ClientHandler(Socket clientSocket, RequestListener requestListener, BlockList blockList, ResponseCache responseCache) {
        this.clientSocket = clientSocket;
        this.requestListener = requestListener;
        this.blockList = blockList;
        this.responseCache = responseCache;
    }

    @Override
    public void run() {
        String threadName = Thread.currentThread().getName();
        try (Socket socket = this.clientSocket;
            InputStream rawIn = socket.getInputStream();
            OutputStream rawOut = socket.getOutputStream();
            BufferedReader reader = new BufferedReader(new InputStreamReader(rawIn, StandardCharsets.UTF_8))) {

            String requestLine = reader.readLine();
            if (requestLine == null || requestLine.isEmpty()) {
                return;
            }
            String[] parts = requestLine.split(" ", 3);
            if (parts.length < 3) {
                System.out.println("Invalid Request Line");
                return;
            }
            String method = parts[0];
            String target = parts[1];
            String httpVer = parts[2];

            Map<String, String> headers = readHeaders(reader);
            String host = "";
            int port = -1;
            String path = "";

            if (method.equalsIgnoreCase("CONNECT")) {
                String[] hp = target.split(":", 2);
                host = hp[0];
                port = hp.length == 2 ? Integer.parseInt(hp[1]) : CONNECT_DEFAULT_PORT;
            } else {
                if (target.startsWith(HTTPS_PREFIX) || target.startsWith(HTTP_PREFIX)) {
                    boolean https = target.startsWith(HTTPS_PREFIX);
                    int prefixLen = https ? HTTPS_PREFIX_LEN : HTTP_PREFIX_LEN;
                    String url = target.substring(prefixLen);
                    int slashIndex = url.indexOf('/');
                    String hostPart = slashIndex != -1 ? url.substring(0, slashIndex) : url;
                    path = slashIndex != -1 ? url.substring(slashIndex) : "/";
                }
            }
        }
    }

```

```

        String[] hp = hostPart.split(":", 2);
        host = hp[0];
        port = hp.length == 2 ? Integer.parseInt(hp[1]) : (https ? CONNECT_DEFAULT_PORT : HTTP_DEFAULT_PORT);
    } else if (target.startsWith("/")) {
        path = target;
        String hostHeader = headers.get("host");
        if (hostHeader == null) {
            System.out.println("No Host Header Present; cannot route request");
            return;
        }
        hostHeader = hostHeader.trim();
        String[] hp = hostHeader.split(":", 2);
        host = hp[0];
        port = hp.length == 2 ? Integer.parseInt(hp[1]) : HTTP_DEFAULT_PORT;
    } else {
        System.out.println("Unrecognized target form: " + target);
        return;
    }
}

System.out.println("ROUTE => " + method + " " + host + ":" + port + " " + path);
long startTime = System.currentTimeMillis();
String clientAddr = clientSocket.getRemoteSocketAddress().toString();

if (blockList != null && blockList.isBlocked(host, path)) {
    sendBlockedResponse(rawOut);
    if (requestListener != null) {
        long duration = System.currentTimeMillis() - startTime;
        requestListener.onRequest(new RequestRecord(Instant.now(), method, host, port, path, duration, true, duration, RequestRecord.SOURCE_BLOCKED));
    }
    System.out.println("BLOCKED => " + host + (path.isEmpty() ? "" : path));
    return;
}

String cacheKey = method.equalsIgnoreCase("GET") ? ResponseCache.cacheKey(host, port, path) : null;
if (method.equalsIgnoreCase("GET") && responseCache != null) {
    CacheEntry cached = responseCache.get(cacheKey);
    if (cached != null) {
        sendCachedResponse(rawOut, cached);
        rawOut.flush();
        if (requestListener != null) {
            long duration = System.currentTimeMillis() - startTime;
            requestListener.onRequest(new RequestRecord(Instant.now(), method, host, port, path, duration, false, duration, RequestRecord.SOURCE_CACHE));
        }
        System.out.println("CACHE HIT => " + host + path + " (" + (System.currentTimeMillis() - startTime) + "ms)");
    }
}

```

```

        return;
    }
}

if (method.equalsIgnoreCase("CONNECT")) {
    runConnectTunnel(rawIn, rawOut, host, port, method, path, clientAddr);
    return;
}

fetchFromOrigin(rawOut, reader, method, path, httpVer, headers, host, port, startTi
} catch (IOException e) {
    System.err.println "[" + threadName + "] ClientHandler Error: " + e.getMessage());
}
}

private static Map<String, String> readHeaders(BufferedReader reader) throws IOException {
    Map<String, String> headers = new HashMap<>();
    String line;
    while ((line = reader.readLine()) != null && !line.isEmpty()) {
        int colonIndex = line.indexOf(':');
        if (colonIndex > 0) {
            String key = line.substring(0, colonIndex).trim().toLowerCase();
            String value = line.substring(colonIndex + 1).trim();
            headers.put(key, value);
        }
    }
    return headers;
}

private void sendBlockedResponse(OutputStream rawOut) throws IOException {
    byte[] bodyBytes = FORBIDDEN_BODY.getBytes(StandardCharsets.UTF_8);
    rawOut.write(("HTTP/1.1 403 Forbidden\r\nConnection: close\r\nContent-Type: text/plain\r\n"
        + bodyBytes.length + "\r\n\r\n").getBytes(StandardCharsets.ISO_8859_1));
    rawOut.write(bodyBytes);
    rawOut.flush();
}

private void runConnectTunnel(InputStream rawIn, OutputStream rawOut, String host, int port) {
    try (Socket serverSocket = new Socket(host, port)) {
        serverSocket.setSoTimeout(0);
        InputStream serverIn = serverSocket.getInputStream();
        OutputStream serverOut = serverSocket.getOutputStream();
        rawOut.write(CONNECTION_ESTABLISHED.getBytes(StandardCharsets.ISO_8859_1));
        rawOut.flush();
        Thread clientToServer = new Thread(() -> copyStream(rawIn, serverOut, serverSocket));
        Thread serverToClient = new Thread(() -> copyStream(serverIn, rawOut, null), "serverToClient");
        clientToServer.start();
    }
}

```

```

        serverToClient.start();
        if (requestListener != null) {
            requestListener.onRequest(new RequestRecord(Instant.now(), method, host, port,
                false, null, RequestRecord.SOURCE_TUNNEL));
        }
        try {
            clientToServer.join();
            serverToClient.join();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}

private static void copyStream(InputStream in, OutputStream out, Socket shutdownOutputOnClose) {
    byte[] buf = new byte[TUNNEL_BUFFER_SIZE];
    try {
        int n;
        while ((n = in.read(buf)) != -1) {
            out.write(buf, 0, n);
            out.flush();
        }
        if (shutdownOutputOnClose != null) {
            shutdownOutputOnClose.shutdownOutput();
        }
    } catch (IOException ignored) {
    }
}

private void fetchFromOrigin(OutputStream rawOut, BufferedReader reader, String method, String path,
    Map<String, String> headers, String host, int port, long startTime) {
    try (Socket serverSocket = new Socket(host, port)) {
        serverSocket.setSoTimeout(ORIGIN_SO_TIMEOUT_MS);
        InputStream serverIn = serverSocket.getInputStream();
        OutputStream serverOut = serverSocket.getOutputStream();
        writeForwardRequest(serverOut, method, path, httpVer, headers, host);
        HttpResponseReader.Response response = HttpResponseReader.readResponse(serverIn);
        if (response == null) {
            return;
        }
        sendResponse(rawOut, response);
        rawOut.flush();
        long duration = System.currentTimeMillis() - startTime;
        if (requestListener != null) {
            requestListener.onRequest(new RequestRecord(Instant.now(), method, host, port,
                false, duration, RequestRecord.SOURCE_ORIGIN));
        }
    }
}

```

```

        System.out.println("ORIGIN => " + host + path + " (" + duration + " ms)");
        maybeStoreInCache(response, cacheKey, host, path, method);
    }
}

private static void writeForwardRequest(OutputStream serverOut, String method, String path,
                                       Map<String, String> headers, String host) throws IOException {
    serverOut.write((method + " " + path + " " + httpVer + "\r\n").getBytes(StandardCharsets.UTF_8));
    for (Map.Entry<String, String> entry : headers.entrySet()) {
        String key = entry.getKey();
        if (key.equalsIgnoreCase("proxy-connection") || key.equalsIgnoreCase("connection")) {
            continue;
        }
        String value = entry.getValue();
        serverOut.write((key.equalsIgnoreCase("host") ? "Host: " + value : key + ": " + value) + "\r\n".getBytes(StandardCharsets.ISO_8859_1));
    }
    if (!headers.containsKey("host")) {
        serverOut.write(("Host: " + host + "\r\n").getBytes(StandardCharsets.ISO_8859_1));
    }
    serverOut.write("Connection: close \r\n\r\n".getBytes(StandardCharsets.ISO_8859_1));
    serverOut.flush();
}

private void maybeStoreInCache(HttpResponseReader.Response response, String cacheKey, String host, String path) {
    if (!method.equalsIgnoreCase("GET") || responseCache == null || cacheKey == null || !isCacheable(response)) {
        return;
    }
    long expiry = responseCache.computeExpiry(response.headers);
    if (expiry <= 0) {
        System.out.println("CACHE SKIP (no-store/private) => " + host + path);
        return;
    }
    List<String> storeHeaders = new ArrayList<>(response.headerLines);
    normalizeHeadersForCache(storeHeaders, response.body.length);
    responseCache.put(cacheKey, new CacheEntry(response.statusLine, storeHeaders, response.body));
    System.out.println("CACHE STORE => " + host + path);
}

private static void sendCachedResponse(OutputStream out, CacheEntry entry) throws IOException {
    out.write((entry.getStatusLine() + "\r\n").getBytes(StandardCharsets.ISO_8859_1));
    for (String h : entry.getHeaders()) {
        out.write((h + "\r\n").getBytes(StandardCharsets.ISO_8859_1));
    }
    out.write("\r\n".getBytes(StandardCharsets.ISO_8859_1));
    out.write(entry.getBody());
}

```

```

private static void sendResponse(OutputStream out, HttpServletResponse.Response response) {
    out.write((response.statusLine + "\r\n").getBytes(StandardCharsets.ISO_8859_1));
    for (String h : response.headerLines) {
        out.write((h + "\r\n").getBytes(StandardCharsets.ISO_8859_1));
    }
    out.write("\r\n".getBytes(StandardCharsets.ISO_8859_1));
    out.write(response.body);
}

private static boolean isSuccessStatus(String statusLine) {
    if (statusLine == null || statusLine.length() < 12) {
        return false;
    }
    int firstSpace = statusLine.indexOf(' ');
    if (firstSpace < 0) {
        return false;
    }
    int secondSpace = statusLine.indexOf(' ', firstSpace + 1);
    if (secondSpace < 0) {
        secondSpace = statusLine.length();
    }
    return STATUS_CODE_200.equals(statusLine.substring(firstSpace + 1, secondSpace));
}

private static void normalizeHeadersForCache(List<String> headerLines, int bodyLength) {
    Iterator<String> it = headerLines.iterator();
    while (it.hasNext()) {
        String line = it.next().toLowerCase();
        if (line.startsWith("transfer-encoding:") || line.startsWith("content-length:")) {
            it.remove();
        }
    }
    headerLines.add("Content-Length: " + bodyLength);
}
}

```

2.9 ManagementConsole.java

Swing JFrame implementing RequestListener. Tabs: **Requests** — table (Time, Method, Host, Path, Client, Status, Time (ms), Source), Clear and Block-host/path buttons; **Block list** — JList of rules, Add text field and Remove selected; **Cache** — table of cache snapshot (URL, Size, Expires, Last access), Refresh and 2s timer. onRequest() runs on EDT and appends a row with duration and source; Source column uses a custom renderer (Cache=green, Origin=blue, Blocked=red, Tunnel=gray). buildTablePanel, buildBlockPanel, buildCachePanel construct each tab; refreshCacheModel/refreshBlockListModel sync UI with shared cache/block list.

```
// Swing UI: request log with timing/source, block list management, and cache view.  
// Implements RequestListener; refreshes cache tab on a timer.
```

```
import javax.swing.*;  
import javax.swing.border.EmptyBorder;  
import javax.swing.table.DefaultTableCellRenderer;  
import javax.swing.table.DefaultTableModel;  
import java.awt.*;  
import java.util.ArrayList;  
import java.util.List;  
  
public class ManagementConsole extends JFrame implements RequestListener {  
  
    private static final String[] COLUMNS = { "Time", "Method", "Host", "Path", "Client", "Stat  
    private static final String[] CACHE_COLUMNS = { "URL", "Size", "Expires", "Last access" };  
    private static final int MAX_ROWS = 2000;  
    private static final int CACHE_REFRESH_MS = 2000;  
    private static final int SOURCE_COLUMN_INDEX = 7;  
  
    private static final Font UI_FONT = new Font(Font.SANS_SERIF, Font.PLAIN, 13);  
    private static final Font TABLE_FONT = new Font(Font.MONOSPACED, Font.PLAIN, 12);  
    private static final int ROW_HEIGHT = 22;  
    private static final Color ROW_ALT = new Color(0xf8f8f8);  
    private static final Color GRID = new Color(0xdddddd);  
    private static final Color SOURCE_CACHE_COLOR = new Color(0x0d6b0d);  
    private static final Color SOURCE_ORIGIN_COLOR = new Color(0x0066aa);  
    private static final Color SOURCE_BLOCKED_COLOR = new Color(0xaa2222);  
    private static final Color SOURCE_TUNNEL_COLOR = new Color(0x666666);  
  
    private final BlockList blockList;  
    private final ResponseCache responseCache;  
    private final DefaultTableModel tableModel;  
    private final DefaultTableModel cacheTableModel;  
    private final List<RequestRecord> requestRecords = new ArrayList<>();  
    private final DefaultListModel<String> blockListModel = new DefaultListModel<>();  
    private final JTable requestTable;  
  
    public ManagementConsole(BlockList blockList) {  
        this(blockList, null);  
    }  
  
    public ManagementConsole(BlockList blockList, ResponseCache responseCache) {  
        super("Proxy Management Console");  
        this.blockList = blockList;  
        this.responseCache = responseCache;  
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);  
        setSize(1100, 660);  
    }  
}
```

```

setLocationRelativeTo(null);
try {
    UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
} catch (Exception ignored) {
}
setFont(UI_FONT);

tableModel = new DefaultTableModel(COLUMNS, 0) {
    @Override
    public boolean isCellEditable(int row, int column) {
        return false;
    }
};

requestTable = new JTable(tableModel);
requestTable.setFont(TABLE_FONT);
requestTable.setRowHeight(ROW_HEIGHT);
requestTable.setSelectionModel(ListSelectionModel.SINGLE_SELECTION);
requestTable.setAutoCreateRowSorter(true);
requestTable.setShowGrid(true);
requestTable.setGridColor(GRID);
requestTable.getTableHeader().setFont(TABLE_FONT);
requestTable.getTableHeader().setReorderingAllowed(false);
setRequestTableColumnWidths(requestTable);
requestTable.setDefaultRenderer(Object.class, new SourceColorRenderer());

JScrollPane tableScroll = new JScrollPane(requestTable);
tableScroll.setBorder(new EmptyBorder(0, 0, 0, 0));
cacheTableModel = new DefaultTableModel(CACHE_COLUMNS, 0) {
    @Override
    public boolean isCellEditable(int row, int column) {
        return false;
    }
};

JPanel tablePanel = buildTablePanel(tableScroll);
JPanel blockPanel = buildBlockPanel();
JPanel cachePanel = buildCachePanel();

JTabbedPane tabs = new JTabbedPane();
tabs.setFont(UI_FONT);
tabs.addTab("Requests", tablePanel);
tabs.addTab("Block list", blockPanel);
tabs.addTab("Cache", cachePanel);
add(tabs, BorderLayout.CENTER);

refreshBlockListModel();
if (responseCache != null) {
    refreshCacheModel();
}

```

```

        new Timer(CACHE_REFRESH_MS, e -> refreshCacheModel()).start();
    }
}

private static void setRequestTableColumnWidths(JTable table) {
    int[] widths = { 88, 58, 180, 220, 130, 72, 72, 72 };
    for (int i = 0; i < widths.length; i++) {
        table.getColumnModel().getColumn(i).setPreferredWidth(widths[i]);
    }
}

private JPanel buildTablePanel(JScrollPane tableScroll) {
    JPanel toolbar = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 8));
    toolbar.setBorder(new EmptyBorder(6, 8, 6, 8));
    JButton clearBtn = new JButton("Clear");
    clearBtn.addActionListener(e -> clear());
    toolbar.add(clearBtn);
    JButton blockHostBtn = new JButton("Block selected host");
    blockHostBtn.addActionListener(e -> blockSelectedHost());
    toolbar.add(blockHostBtn);
    JButton blockPathBtn = new JButton("Block selected host + path");
    blockPathBtn.addActionListener(e -> blockSelectedHostPath());
    toolbar.add(blockPathBtn);
    toolbar.add(new JLabel(" - Time (ms) and Source prove cache speed: Cache = from cache"));
    JPanel panel = new JPanel(new BorderLayout());
    panel.setBorder(new EmptyBorder(8, 8, 8, 8));
    panel.add(tableScroll, BorderLayout.CENTER);
    panel.add(toolbar, BorderLayout.SOUTH);
    return panel;
}

private JPanel buildBlockPanel() {
    JPanel panel = new JPanel(new BorderLayout(10, 10));
    panel.setBorder(BorderFactory.createCompoundBorder(new EmptyBorder(8, 8, 8, 8), BorderLayoutFactory.createCompoundBorder(
        new EmptyBorder(8, 8, 8, 8), BorderLayout.SOUTH)));
    JList<String> blockJList = new JList<>(blockListModel);
    blockJList.setFont(TABLE_FONT);
    blockJList.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
    JPanel toolbar = new JPanel(new FlowLayout(FlowLayout.LEFT, 8, 8));
    toolbar.setBorder(new EmptyBorder(0, 0, 6, 0));
    JTextField addField = new JTextField(28);
    addField.setFont(UI_FONT);
    addField.setToolTipText("URL or host, e.g. https://youtube.com/ or example.com/ads");
    JButton addBtn = new JButton("Add");
    addBtn.addActionListener(e -> {
        String rule = addField.getText().trim();
        if (!rule.isEmpty()) {
            String normalized = blockList.add(rule);
        }
    });
}

```

```

        if (normalized != null && !blockListModel.contains(normalized)) {
            blockListModel.addElement(normalized);
        }
        addField.setText("");
    }
});
JButton removeBtn = new JButton("Remove selected");
removeBtn.addActionListener(e -> {
    int i = blockJList.getSelectedIndex();
    if (i >= 0) {
        blockList.remove(blockListModel.get(i));
        blockListModel.remove(i);
    }
});
toolbar.add(new JLabel("Add:"));
toolbar.add(addField);
toolbar.add(addBtn);
toolbar.add(removeBtn);
panel.add(new JScrollPane(blockJList), BorderLayout.CENTER);
panel.add(toolbar, BorderLayout.SOUTH);
return panel;
}

private JPanel buildCachePanel() {
    JTable cacheTable = new JTable(cacheTableModel);
    cacheTable.setFont(TABLE_FONT);
    cacheTable.setRowHeight(ROW_HEIGHT);
    cacheTable.setAutoCreateRowSorter(true);
    cacheTable.setShowGrid(true);
    cacheTable.setGridColor(GRID);
    cacheTable.getTableHeader().setFont(TABLE_FONT);
    cacheTable.getColumnModel().getColumn(0).setPreferredWidth(380);
    cacheTable.getColumnModel().getColumn(1).setPreferredWidth(72);
    cacheTable.getColumnModel().getColumn(2).setPreferredWidth(100);
    cacheTable.getColumnModel().getColumn(3).setPreferredWidth(100);
    cacheTable.setDefaultRenderer(Object.class, new DefaultTableCellRenderer() {
        @Override
        public Component getTableCellRendererComponent(JTable t, Object value, boolean selected,
            Component c = super.getTableCellRendererComponent(t, value, selected, focus, row)
            if (!selected) {
                c.setBackground(row % 2 == 0 ? Color.WHITE : ROW_ALT);
            }
            return c;
        }
    });
    JPanel toolbar = new JPanel(new FlowLayout(FlowLayout.LEFT, 8, 8));
    JButton refreshBtn = new JButton("Refresh");

```

```

refreshBtn.addActionListener(e -> refreshCacheModel());
toolbar.add(refreshBtn);
toolbar.add(new JLabel(" Only HTTP GET is cached; use http://... for sites like neverssl
JPanel panel = new JPanel(new BorderLayout(10, 10));
panel.setBorder(new EmptyBorder(8, 8, 8, 8));
panel.add(new JScrollPane(cacheTable), BorderLayout.CENTER);
panel.add(toolbar, BorderLayout.SOUTH);
return panel;
}

private final class SourceColorRenderer extends DefaultTableCellRenderer {
    @Override
    public Component getTableCellRendererComponent(JTable t, Object value, boolean selected,
        Component c = super.getTableCellRendererComponent(t, value, selected, focus, row, c);
    if (!selected) {
        c.setBackground(row % 2 == 0 ? Color.WHITE : ROW_ALT);
    }
    if (col == SOURCE_COLUMN_INDEX && value != null) {
        String s = value.toString();
        if (RequestRecord.SOURCE_CACHE.equals(s)) {
            c.setForeground(SOURCE_CACHE_COLOR);
        } else if (RequestRecord.SOURCE_ORIGIN.equals(s)) {
            c.setForeground(SOURCE_ORIGIN_COLOR);
        } else if (RequestRecord.SOURCE_BLOCKED.equals(s)) {
            c.setForeground(SOURCE_BLOCKED_COLOR);
        } else if (RequestRecord.SOURCE_TUNNEL.equals(s)) {
            c.setForeground(SOURCE_TUNNEL_COLOR);
        } else {
            c.setForeground(Color.BLACK);
        }
    } else if (!selected) {
        c.setForeground(Color.BLACK);
    }
    return c;
}

private void refreshCacheModel() {
    if (responseCache == null) {
        return;
    }
    cacheTableModel.setRowCount(0);
    long now = System.currentTimeMillis();
    for (ResponseCache.CacheEntryInfo info : responseCache.getSnapshot()) {
        String expires = info.getExpiryMillis() <= now ? "Expired" : formatDuration((info.g
        String size = formatSize(info.getBodySize());
        String lastAccess = formatDuration((now - info.getLastAccessMillis()) / 1000) + " a

```

```

        cacheTableModel.addRow(new Object[]{ info.getDisplayUrl(), size, expires, lastAccessTime });
    }
}

private static String formatDuration(long seconds) {
    if (seconds < 60) {
        return seconds + "s";
    }
    if (seconds < 3600) {
        return (seconds / 60) + "m " + (seconds % 60) + "s";
    }
    return (seconds / 3600) + "h " + ((seconds % 3600) / 60) + "m";
}

private static String formatSize(int bytes) {
    if (bytes < 1024) {
        return bytes + " B";
    }
    if (bytes < 1024 * 1024) {
        return String.format("%.1f KB", bytes / 1024.0);
    }
    return String.format("%.2f MB", bytes / (1024.0 * 1024));
}

private void refreshBlockListModel() {
    blockListModel.clear();
    for (String rule : blockList.getAll()) {
        blockListModel.addElement(rule);
    }
}

@Override
public void onRequest(RequestRecord record) {
    SwingUtilities.invokeLater(() -> {
        while (tableModel.getRowCount() >= MAX_ROWS) {
            tableModel.removeRow(0);
            requestRecords.remove(0);
        }
        String path = record.getPath();
        if (path.isEmpty()) {
            path = record.getMethod().equalsIgnoreCase("CONNECT") ? "(tunnel)" : "/";
        }
        String status = record.isBlocked() ? "Blocked" : "Forwarded";
        String timeMs = record.getDurationMs() != null ? String.valueOf(record.getDurationMs()) : "-";
        String source = record.getSource() != null ? record.getSource() : "-";
        tableModel.addRow(new Object[]{
            record.getTimeString(),
            path,
            status,
            timeMs,
            source
        });
    });
}

```

```

        record.getMethod(),
        record.getHostPort(),
        path,
        record.getClientAddress(),
        status,
        timeMs,
        source
    });
    requestRecords.add(record);
});
}

private static void setUIFont(Font font) {
    for (Object key : UIManager.getLookAndFeelDefaults().keySet()) {
        if (key.toString().endsWith(".font")) {
            UIManager.put(key, font);
        }
    }
}

private void blockSelectedHost() {
    RequestRecord r = getSelectedRecord();
    if (r == null) {
        return;
    }
    String rule = r.getHost().toLowerCase();
    if (!blockList.contains(rule)) {
        String normalized = blockList.add(rule);
        if (normalized != null && !blockListModel.contains(normalized)) {
            blockListModel.addElement(normalized);
        }
    }
}

private void blockSelectedHostPath() {
    RequestRecord r = getSelectedRecord();
    if (r == null) {
        return;
    }
    String path = r.getPath();
    if (path.isEmpty() || "(tunnel)".equals(path)) {
        JOptionPane.showMessageDialog(this, "CONNECT requests have no path; use \"Block sel
        return;
    }
    String pathPart = path.startsWith("/") ? path.substring(1) : path;
    String rule = (r.getHost() + "/" + pathPart).toLowerCase();
    if (!blockList.contains(rule)) {

```

```

        String normalized = blockList.add(rule);
        if (normalized != null) {
            blockListModel.addElement(normalized);
        }
    }
}

private RequestRecord getSelectedRecord() {
    int viewRow = requestTable.getSelectedRow();
    if (viewRow < 0) {
        JOptionPane.showMessageDialog(this, "Select a request row first.");
        return null;
    }
    int modelRow = requestTable.convertRowIndexToModel(viewRow);
    if (modelRow >= requestRecords.size()) {
        return null;
    }
    return requestRecords.get(modelRow);
}

private void clear() {
    tableModel.setRowCount(0);
    requestRecords.clear();
}
}

```